

- Q1. (a) (i) $O(n^{0.7})$
(ii) $O(2^{1.5n})$
(iii) $O(n(n!))$
(iv) $O(n^2)$
(v) $O(\log n)$
(vi) $O(n \log \log n)$
- (b) $\exists c=2000, n_0=1 \bullet \forall n > n_0 \bullet$
 $n \geq 1$
 $n^{0.01} \geq 1$
 $n^{10.01} \geq n^{10}$
 $n^{10} \leq n^{10.01}$
 $2000n^{10} \leq 2000n^{10.01}$
 $2000n^{10} \leq cn^{10.01}.$
- (c) Need to prove $[\exists c, n_0 \bullet \forall n > n_0 \bullet 2000n^{10} \geq cn^{10.01}]$ is wrong.
Equivalent to prove for $[\forall c, n_0 \bullet \exists n > n_0 \bullet 2000n^{10} < cn^{10.01}]$
For any c, n_0 , take $n = \max(n_0, \lceil (2000/c)^{100} \rceil) + 1$, we have
 $n > (2000/c)^{100}$
 $n^{0.01} > 2000/c$
 $2000 < cn^{0.01}.$
 $2000n^{10} < cn^{10.01}.$
- (d) $\forall c \bullet (\exists) \text{ take } n_0 = \max(\lceil (64^{64})/(63! \cdot c) \rceil, 64) \bullet \forall n > n_0$
we have $(64^{64})/(63! \cdot c) < n$ replacing n_0 by $(64^{64})/(63! \cdot c)$
 $\Rightarrow (64^{64}) < c(63!)n$ rewriting
 $\Rightarrow (64^{64}) (64^{n-64}) < c(63!)n(64^{n-64})$ multiplying 64^{n-64} to both sides
 $\Rightarrow 64^n < c(63!)n(64)(65)\dots(n-1)$ replacing 64^{n-64} by $(64)(65)\dots(n-1)$
 $\Rightarrow 64^n < c(1)(2)\dots(63)(64)(65)\dots(n-1)(n)$ rewriting
 $\Rightarrow 2^{6n} < c(n!)$ rewriting

- Q2. (a) $T(n) = 3T(n/3) + n$
- (i) By substitution (Mathematical Induction)
Claim: $T(n) \leq cn \log n$, for some $c \geq 1 / \log 3, n > 3$.
 $T(n) = 3T(n/3) + n$
 $\leq 3c(n/3)\log(n/3) + n$
 $= cn(\log n - \log 3) + n$
 $= cn \log n - cn \log 3 + n$
 $\leq cn \log n$
- (ii) By recursive method (Expansion).
 $T(n) = 3T(n/3) + n$
 $= 3[3T(n/3^2) + (n/3)] + n$
 $= 3^2T(n/3^2) + 2n$
 $= 3^kT(n/3^k) + kn$
 $= nT(1) + n \log_3 n$, setting $k = \log_3 n$
 $= \Theta(n \log n)$

- (b)(i) Solvng $T(n) = 3 T(n/3) + n \log n$ by master method.
 $a=3, b=3, \log_b a=1, f(n)=n \log n$.
 $\exists k=1$ •
 $f(n) = n \log n = [n^{(\log_3 3)}] \log^1 n = [n^{(\log_b a)}] \log^k n$
 $f(n) = \Theta(n^{(\log_b a)} \log^k n)$.
Case 2 applies. $T(n) = \Theta(n^{(\log_b a)} \log^{k+1} n) = \Theta(n \log^2 n)$.
- (ii) Solvng $T(n) = 3 T(n/3) + n^{0.5} \log n$ by master method.
 $a=3, b=3, \log_b a=1, f(n)=n^{0.5} \log n$.
 $\exists \varepsilon = 1/4 > 0$ •
 $f(n) = n^{0.5} \log n < n^{3/4} = n^{(\log_3 3 - 1/4)} = n^{(\log_b a - \varepsilon)}$
Then, $f(n) = O(n^{(\log_b a - \varepsilon)})$
Case One applies. $T(n) = \Theta(n^{(\log_b a)}) = \Theta(n)$.
- (iii) Solvng $T(n) = 3 T(n/3) + n^{1.5} \log n$ by master method.
 $a=3, b=3, \log_b a=1, f(n)=n^{1.5} \log n$.
 $\exists \varepsilon=1/2$ such that
 $f(n) = n^{1.5} \log n > n^{1.5} = n^{(\log_3 3 + 1/2)} = n^{(\log_b a + \varepsilon)}$
 $f(n) = O(n^{(\log_b a + \varepsilon)})$.
 $\exists c=(1/3^{0.5}) < 1$ such that
 $a f(n/b) = 3 (n/3)^{1.5} \log (n/3) < (3/3^{1.5}) n^{1.5} \log n = c n^{1.5} \log n = c f(n)$
Case Three applies. $T(n) = \Theta(f(n)) = \Theta(n^{1.5} \log n)$.
- (c) Given $\exists c < 1, n_0 > 0 \bullet \forall n > n_0 \bullet a f(n/b) < c f(n)$.
Then, $\exists c < 1, n_0 > 0 \bullet \forall n > n_0 \bullet a f(n/b) < c f(n) \Rightarrow a f(n/b) \leq c f(n)$.
Then, $\exists c < 1, n_0 > 0 \bullet \forall n > n_0 \bullet a f(n/b) \leq c f(n)$.

Given $\exists c < 1, n_0 > 0 \bullet \forall n > n_0 \bullet a f(n/b) \leq c f(n)$.
Then, $\exists c' = (1+c)/2 < 1, c < c', n_0 > 0 \bullet \forall n > n_0 \bullet a f(n/b) \leq c f(n) < c' f(n)$.
Then, $\exists c' < 1, n_0 > 0 \bullet \forall n > n_0 \bullet a f(n/b) < c' f(n)$. QED

- Q3. (a) Construct an array of pointers where index i is considered as vertex i , and inside the linked list of nodes being pointed by index i , a node labeled as j represents the edge (i, j) in the graph.
- (b) On removing an edge (i, j) .
Traverse the linked list pointed by index i and search for node j . Remove node j from the linked list.
Traverse the linked list pointed by index j and search for node i . Remove node i from the linked list.
- On adding an edge (i, j) .
Suppose the edge (i, j) does not exist, and therefore, no checking is needed.
Go to the linked list of index i , and add a node j in the front position.
Go to the linked list of index j , and add a node i in the front position.
- c) $O(n)$ and $O(1)$, respectively.
 $O(n)$: The time to search for the indices is $O(1)$, and the time to search the two linked list is $O(n)$.
 $O(1)$: The time to search for the indices is $O(1)$, and the time to add in the front position is $O(1)$.

- d) $O(d)$ and $O(1)$, respectively.
 In the first case, the length of a linked list pointed by index i is the number of neighbours of vertex i . Therefore, the time to search the two linked list is $O(d)$.
 The second case has no change.
- e) Construct a sequence of cells as follows. (1) The first cell represents vertex 1, ..., the n th cell represents vertex n . (2) Add a delimiter -1. (3) After the delimiter, place the node labels of the first linked list in order, then those of the second linked list, etc. Each of the first n cells in the sequence, which represent the vertices, will store the position of the cell where its adjacency list starts. If there is no adjacency list, the cell will store -2.
- f) $O(n^2)$. A node has at most $n-1$ neighbours.

- Q4. (a) $m_{ii} = 0$
- (b) $m_{ij} = \min_{k \in [i,j-1]} \{m_{ik} + m_{k+1j} + p_{i-1}p_kp_j\}$, where $j = i+2$.
- (c) $m_{ij} = \min_{k \in [i,j-1]} \{m_{ik} + m_{k+1j} + p_{i-1}p_kp_j\}$, for $j \neq i$.
 $m_{ii} = 0$.
- (d) Algorithm $m(i, j)$
 if $i=j$
 return 0;
 min = infinity;
 for $k=i$ to $j-1$
 temp = $m(i, k) + m(k+1, j) + p_{i-1}p_kp_j$;
 if temp < min
 min = temp;
 return min;
- (e) Construct an $N \times N$ table, $M[1..N, 1..N]$.
 for $i=1$ to N
 $M[i, i] = 0$;

 for $r=1$ to $N-1$
 for $i=1$ to $N-r$
 $j=i+r$;
 min = infinity;
 for $k=i$ to $j-1$
 temp = $M(i, k) + M(k+1, j) + p_{i-1}p_kp_j$;
 if temp < min
 min = temp;
 $M[i, j] = \text{min}$;

- Q5. (a) Base case: $N=1$.
 $p=r$ is true in the first if-condition. Selection() returns without doing anything.
 As there is only one element, the sorted property is implied.
 Therefore, the statement is true for the base case.

Assume the statement is true for $N=k \geq 1$.
 Consider the statement for $N=k+1$.
 After the execution of line 3, $A[j]$ = the maximum element.

Then, line 4 swap the largest element to $A[r]$.

By inductive assumption, $\text{Selection}(A, p, r-1)$ will sort the array $A[p \dots r-1]$ in ascending order. Together with the maximum element at $A[r]$, $A[p \dots r]$ is sorted ascendingly.

Result follows from induction.

- (b) (i) $O(1)$ (ii) $O(N)$ (iii) $O(1)$ (iv) $O(1)$
- (c) The recurrence is $T(n) = T(n-1) + n$.
 $T(n) = T(n-1) + n = T(n-2) + n + (n-1) = T(n-3) + n + (n-1) + (n-2) = \dots$
 $= T(1) + n + (n-1) + (n-2) + \dots + 2 = O(n^2)$
- (d) The elements are arranged as 2, 3, ..., N, 1. Then, 1 will be moved for $N-1$ times.